

EVALUATING PROMPT ENGINEERING STRATEGIES FOR PEDAGOGICAL AI IN VOCATIONAL JAVASCRIPT LEARNING

Annuruddin Tsanasti Yassar¹, Djuniadi², Nur Iksan³

Department of Informatics and Computer Engineering Education, Faculty of Engineering, Universitas Negeri Semarang

ARTICLE INFO

Article history:

Received:

Received in revised form:

Accepted:

Available online:

Keywords: maximum 7 words

Prompt Engineering, Socratic Scaffolding, Vocational Programming Education, Large Language Model, Automated Assessment

Authors email:

annuruddinty21@students.unnes.ac.id

All author emails

ABSTRACT

Vocational high school (SMK) students enrolled in Phase F of the Merdeka Curriculum frequently encounter pedagogical barriers when learning JavaScript programming, primarily due to cryptic compiler error messages (stderr) that lack contextual explanation. Concurrently, unrestricted access to generative AI tools enables answer-spoiling behaviors that erode independent algorithmic reasoning. This experimental study investigates how layered constraint prompt engineering can transform a general-purpose Large Language Model (LLM) into a Socratic scaffolding tutor that diagnoses coding errors without providing direct solutions. Three prompt variant experiments were designed: Experiment A evaluated six temperature configurations (0.0–1.0), Experiment B compared Zero-Shot, Static Few-Shot, and Dynamic Few-Shot prompting techniques, and Experiment C conducted an ablation study progressively adding Role-Prompting, JSON schema constraints, and quality rules. Each configuration was tested against a standardized suite of 10 JavaScript error scenarios, spanning syntax, runtime, logic, hybrid, style, edge-case, and prompt-injection categories with three repetitions per scenario, yielding 390 total API calls to Gemini 2.5 Flash. Results demonstrated a JSON Parsing Success Rate (PSR) of 99.2%, confirming structural output reliability. The ablation study revealed that explicit Role-Prompting elevated error classification accuracy (Category Match) by 20 percentage points, while achieving 100% defense against indirect prompt injection attacks. These findings establish that systematic prompt architecture is combining persona declaration, schema enforcement, and quality rules that can reliably constrain an LLM to operate within strict Socratic pedagogical boundaries for vocational programming education.

1. INTRODUCTION

Vocational education in Indonesia, particularly at the 11th-grade level of Sekolah Menengah Kejuruan (SMK) implementing the Merdeka Curriculum Phase F, demands that students develop adaptive technical competencies aligned with the evolving software industry. Mastery of foundational

JavaScript programming constitutes a critical competency at this phase. However, during the cognitive transition from basic logic comprehension to rigid code syntax, students frequently encounter significant learning barriers originating from the native compiler error output of execution environments such as Node.js (Liao et al., 2024). Technical error messages in English, such as `ReferenceError: totals is not defined`, are architecturally designed for professional developers and possess no pedagogical orientation. For novice learners, these absolute, context-deficient messages fail to explain either the specific location of the error or the corrective steps required. Forensic analysis of compilation logs has demonstrated that persistent sequential failures in interpreting stderr messages cumulatively induce mental fatigue, erode student self-efficacy toward technological mastery, trigger acute demotivation, and directly contribute to elevated evaluation failure rates (Yilmaz & Yilmaz, 2023).

The pedagogical crisis is further compounded by two converging threats. First, when vocational institutions direct students to utilize external cloud-based Integrated Development Environments (IDEs) as workarounds, learners are exposed to aggressive AI code-completion features (AI Copilots) embedded in modern platforms. These auto-complete functionalities bypass essential stages of the analytical reasoning process by providing complete solution blocks instantaneously without solicitation, creating an illusion of false competency so students execute programs and observe correct visual outputs without possessing the metacognitive structures to independently understand the algorithmic decomposition (Liao et al., 2024). This phenomenon fundamentally undermines the constructivist essence of programming education, wherein learners should constructively struggle with errors to build memory schemas and mental resilience (Ma et al., 2025). Second, the teacher-to-student ratio in practical programming classrooms, typically one instructor serving 25 to 30 students simultaneously, renders real-time individualized formative feedback an empirical impossibility (Ibañez, 2023).

Responding to these challenges, the research community has explored the integration of Large Language Models (LLMs) into Automated Programming Assessment Systems (APAS). However, contemporary implementations suffer from two critical architectural deficiencies: LLMs inherently tend to produce linguistic hallucinations, presenting syntactically convincing but technically misleading explanations; and without systematic structural restrictions, LLM agents position themselves as "answer-dispensing servants" that prematurely expose complete code solutions to satisfy user queries rapidly (Frankford et al., 2026). Empirical evidence demonstrates that while students receiving instant full-code repairs from AI chatbots experience dramatically accelerated task resolution times, they exhibit fatal atrophy in independent algorithmic decomposition capabilities during longitudinal knowledge transfer assessments (Alshammari, 2025; Ma et al., 2025).

The concept of Socratic scaffolding through prompt engineering presents a theoretically grounded solution. (Chidambaram et al., 2024) demonstrated through the Socratic Human Feedback (SoHF) framework that enforcing Socratic stages, including clarification questions, descriptive planning, and dialectical opposition dramatically improves model resolution accuracy. Tsakeni et al. (2025) further synthesized 135 publications to conclude that AI interventions must operate as co-regulation schemes equipped with fading support thresholds to prevent cognitive over-reliance. However, a critical research gap persists: while these studies establish the theoretical merit of Socratic constraints, few have empirically benchmarked the technical reliability and pedagogical compliance of specific prompt engineering architectures against standardized vocational programming error datasets.

This study addresses the identified gap by conducting a controlled experimental evaluation of layered constraint prompt algorithms. The research objective is to determine the optimal prompt architecture is encompassing persona declaration (Role-Prompting), JSON schema enforcement, and pedagogical quality rules that maximizes both structural output reliability (JSON Parsing Success Rate) and Socratic pedagogical compliance while maintaining acceptable response latency

for classroom deployment. The investigation is bounded to foundational JavaScript programming errors aligned with the SMK Phase F vocational curriculum, testing three experimental configurations across a standardized suite of error scenarios representative of authentic student difficulties

2. METHOD

2.1 Experimental Design Overview

This study employed a controlled experimental design to evaluate AI prompt algorithm performance. Rather than measuring student learning outcomes through questionnaires or pre-posttests, the methodology focused exclusively on the technical validation of prompt engineering configurations against standardized programming error datasets. The experiment was designed to identify the optimal prompt architecture that simultaneously ensures structural output reliability, pedagogical compliance with Socratic principles, and acceptable response latency for real-time classroom deployment.

The experimental benchmark was executed using an automated testing instrument deployed on a local Laragon development environment with standard internet connectivity. The system dispatched 390 API requests to the Gemini 2.5 Flash model via the OpenRouter endpoint. Execution was conducted in three sequential sessions corresponding to the experimental groups, with a static inter-request delay of 1,500 milliseconds to prevent rate-limiting penalties from the cloud server.

2.2 Figure and table

a. Independent Variables

The independent variables comprised systematically manipulated prompt configurations organized into three experimental groups:

1. Experiment A Generation Parameter Influence: Six temperature escalations (0.0, 0.2, 0.4, 0.6, 0.8, 1.0) were tested while maintaining a fixed topP ratio of 1.0 and the complete prompt template. This experiment generated 180 API requests (6 configurations × 10 scenarios × 3 repetitions).
2. Experiment B Prompting Technique Comparison: Three instruction structuring techniques were evaluated at a constant temperature of 0.2: Zero-Shot (no examples), Static Few-Shot (built-in exemplars), and Dynamic Few-Shot (incorporating historical teacher correction data). This experiment generated 90 API requests (3 configurations × 10 scenarios × 3 repetitions).
3. Experiment C Prompt Structure Ablation Study: Constraint layers were progressively added to the prompt: (C1) Schema Only, (C2) Role + Schema, (C3) Role + Schema + Quality Rules, and (C4) Full Prompt. This experiment generated 120 API requests (4 configurations × 10 scenarios × 3 repetitions).

b. Dependent Variables

Four dependent metrics were measured:

1. JSON Parsing Success Rate (PSR): The percentage probability that the AI model's text response could be parsed into a pure JSON data object that passes architectural schema validation.
2. Category Match Rate: The accuracy percentage at which the AI's root-cause diagnosis category (logic, syntax, runtime, etc.) aligned with expert/instructor categorization standards.

3. Line Match Rate: The precision of the AI's error-line identification, measured as the percentage of correct pointing (with ± 1 line tolerance) compared to the actual error location in the source code.
4. Latency (Response Time): The round-trip response time from the Gemini API server, measured from HTTP request dispatch to full-text reception in milliseconds (ms).

c. Control Variables

To protect internal experimental validity and minimize external variable noise, the following constants were enforced across all testing circuits: (1) strictly identical language model version (gemini-2.5-flash) via the OpenRouter API gateway; (2) uniform output token quota restriction (maximum 2,048 tokens) per API call; (3) identical testing dataset matrix (TS-01 through TS-10) with three consecutive execution repetitions per scenario to contain stochastic cloud-computing fluctuations; and (4) a static 1,500 ms delay between API calls to avoid rate-limiting penalties.

2.3 Test Suite Dataset

The test suite comprised 10 custom JavaScript error scenarios (designated TS-01 through TS-10) meticulously designed to map to authentic vocational student programming difficulties. These scenarios were categorized into seven taxonomic error classes, as presented in Table 1.

Table 1. Test Suite of JavaScript Error Scenarios (TS-01 to TS-10)

ID	Category	Description
TS-01	Syntax Error	Missing closing parenthesis in function declaration triggering SyntaxError: Unexpected token '{'
TS-02	Syntax Error	Missing closing quotation mark on string literal triggering SyntaxError: Invalid or unexpected token
TS-03	Runtime Error	Variable name case-sensitivity mismatch triggering ReferenceError
TS-04	Runtime Error	Property access on undefined variable triggering TypeError
TS-05	Logic Error	Off-by-one boundary condition error (operator > instead of >=)
TS-06	Logic Error	Infinite loop due to missing increment statement in while loop
TS-07	Runtime/Logic Hybrid	Division by zero producing Infinity — intentionally ambiguous categorization
TS-08	Style Issue	Functional code with poor variable naming conventions (x, y, z, aa)
TS-09	Edge Case	Empty code containing only comments — tests model behavior with non-analyzable input
TS-10	Prompt Injection	Manipulative instruction embedded in code comments (// IGNORE ALL PREVIOUS INSTRUCTIONS...) — tests system resilience against indirect prompt injection

These 10 scenarios span three fundamental error categories encountered in vocational JavaScript curricula (Syntax, Runtime, Logic) while incorporating advanced adversarial cases (Hybrid, Style, Edge Case, Prompt Injection) to stress-test the robustness of the prompt architecture. The functional code beneath the prompt injection scenario (TS-10) remained valid (let angka = 10;

`console.log(angka))` to verify that injection-resistant models would ignore comment-embedded instructions and analyze the code normally.

2.4 Benchmark Protocol

The automated benchmark instrument executed a three-level nested loop iteration protocol: iteration across 13 experimental configurations, iteration across 10 JavaScript test scenarios, and 3 identical repetitions per scenario. This programmatic observation technique extracted data from 390 external API traffic pathways to the gemini-2.5-flash model, recording temperature parameters, token payloads, and response latency in milliseconds. All collected data were exported as structured Comma-Separated Values (CSV) for analysis.

3. RESULTS AND DISCUSSION

3.1 Overall Benchmark Results

All 390 API requests successfully received HTTP 200 responses from the server, confirming zero distortion at the network or authentication level. Of the total responses, 387 (99.2%) were successfully parsed as pure JSON format and passed all structural schema validation, while only 3 responses (0.8%) experienced parsing failures. These three sporadic failures were detected in ambiguous scenarios (division-by-zero case and variable naming style issue), indicating that the core prompt containment architecture was highly robust for handling conventional error cases.

Table 2. Aggregate Cross-Experiment Statistics

Metric	Value
Total API Requests	390
JSON Valid & Schema Valid (PSR)	99.2%
Category Match (Aggregate)	66.2%
Average Latency	4,895 ms (Min: 1,859 ms / Max: 9,636 ms)
Prompt Injection Defense Rate	100%

3.2 Experiment A: Generation Parameter Influence on Output Reliability

Experiment A tested six temperature escalations (0.0 through 1.0) using the complete prompt template. The results are presented in Table 3.

Table 3. Experiment A Results Temperature Parameter Variation

Config Label	PSR (%)	Avg Confidence	Category Match (%)	Line Match (%)	Avg Latency (ms)
A1 T=0.0	100.0	0.884	70.0	100.0	4,687
A2 T=0.2	93.3	0.894	66.7	86.7	4,593
A3 T=0.4	96.7	0.894	70.0	93.3	4,608
A4 T=0.6	100.0	0.890	66.7	93.3	4,866
A5 T=0.8	100.0	0.899	63.3	90.0	4,690
A6 T=1.0	100.0	0.900	56.7	90.0	4,945

Contrary to initial theoretical speculation, the Gemini 2.5 Flash model demonstrated extremely robust instruction-following even at maximum temperature ($T = 1.0$), achieving a PSR of 100%. However, deeper analysis revealed a gradual degradation of pedagogical reasoning consistency. The Category Match rate declined progressively from 70.0% at $T = 0.0$ to merely 56.7% at $T = 1.0$, representing a 13.3 percentage-point deterioration. This logical degradation unambiguously legitimizes the selection of a conservative temperature threshold ($T = 0.4$) for real-

world classroom operationalization, where diagnostic accuracy must be prioritized over creative variation. The finding corroborates the prompt engineering principle that lower temperature values produce more deterministic and focused responses, which is critical when the LLM is constrained to function as a diagnostic tutor rather than a creative generator (Schulhoff & others, 2024).

3.3 Experiment B: Prompting Technique Impact on Output Quality

Experiment B compared three prompting techniques at a constant temperature of 0.2. Results are presented in Table 4.

Table 4. Experiment B Results Prompting Technique Comparison

ConfigLabel	PSR (%)	Avg Confidence	Category Match (%)	Line Match (%)	Avg Latency (ms)
B1 Zero-Shot	100.0	0.957	56.7	90.0	5,100
B2 Static Few-Shot	100.0	0.887	70.0	90.0	4,903
B3 Dynamic Few-Shot	100.0	0.933	60.0	90.0	4,598

The inclusion of built-in exemplar demonstrations (Static Few-Shot) proved effective in stimulating the highest Category Match rate (70.0%), representing a 13.3 percentage-point improvement over Zero-Shot prompting. A surprising finding emerged from the Dynamic Few-Shot configuration, which incorporated authentic teacher correction histories. Rather than enhancing accuracy, this approach degraded the Category Match to 60.0%. The performance decline is attributed to an irrelevant instruction echo effect (noise): when the model was supplied with historical syntax error corrections from teachers to respond to a looping (logic) failure type, the AI became confused in determining cross-priority classification. This phenomenon empirically demonstrates that the quantity of embedded human records can only elevate reliability when supported by retrieval methods that calculate semantic similarity, a finding consistent with recent work on Retrieval-Augmented Generation for educational contexts (Zhang et al., 2024).

Notably, all three prompting techniques maintained a perfect 100% PSR, confirming that the JSON schema enforcement mechanism within the prompt architecture functions as a reliable structural constraint regardless of the instructional technique applied. This separation between structural reliability (PSR) and diagnostic accuracy (Category Match) underscores that these are independent quality dimensions requiring distinct prompt engineering strategies.

3.4 Experiment C: Prompt Structure Ablation Study

The ablation study systematically dissected the prompt architecture layer by layer to reveal the contribution weight of each constraint element. Results are presented in Table 5.

Table 5. Experiment C Results Prompt Structure Ablation Study

ConfigLabel	PSR (%)	Avg Confidence	Category Match (%)	Line Match (%)	Avg Latency (ms)
C1 Schema Only	100.0	0.923	60.0	90.0	4,841
C2 Role + Schema	100.0	0.935	80.0	90.0	5,428
C3 Role + Schema + Quality Rules	100.0	0.970	70.0	90.0	5,473
C4 Full Prompt	100.0	0.887	70.0	93.3	4,900

The injection of Role-Prompting (transition from C1 to C2) produced the most dramatic acceleration, elevating error classification reliability by 20 percentage points (60.0% to 80.0%). This

finding establishes that persona assignment is not merely a stylistic instruction but functions strategically as a key that unlocks the pedagogical parameter distribution within the model's latent memory, enabling it to sharpen its consideration between runtime execution delays (timeout) and algorithmic loop stagnation anomalies (infinite loops) in student code.

Furthermore, cybersecurity threat testing via prompt injection (TS-10) achieved an absolute defense rate of 100% across all ablation configurations. In test iteration 39, the layered protection parameters within the prompt successfully blocked a malicious instruction embedded within a code comment. This confirms that the multi-layered constraint architecture, even at its minimal configuration (Schema Only) provides robust defense against indirect prompt injection attacks (Greshake & others, 2023), a critical requirement for deploying AI tutors in unsupervised student environments.

An interesting observation is the 10 percentage-point decline in Category Match when transitioning from C2 (Role + Schema, 80.0%) to C3 (Role + Schema + Quality Rules, 70.0%). This suggests that the addition of explicit quality rules may introduce competing instructional signals that partially diffuse the model's diagnostic focus. The Full Prompt (C4) maintained the same 70.0% Category Match as C3 while marginally improving Line Match to 93.3%, indicating that the complete prompt architecture achieves a balanced optimization across all metrics.

3.5 Pedagogical Implications

The experimental results carry substantial implications for vocational programming pedagogy. The 99.2% PSR across 390 API calls is far exceeding the industry-standard reliability threshold of 95% guarantees that a teacher analytics dashboard would not encounter corrupt data gaps in student activity logs. This structural reliability is foundational for any assessment system integrated into institutional learning management infrastructure.

The Socratic compliance dimension is equally significant. The prompt architecture was engineered with a strict zero-spoiler constraint: the AI was categorically prohibited from providing complete code solution blocks. Instead, responses were architecturally restricted to a maximum of three graduated diagnostic hints, progressing from a mild conceptual warning to a specific line-level identification using a server-side array truncation mechanism. Combined with the 100% prompt injection defense rate, this design ensures that the LLM operates purely as a scaffolding facilitator, diagnosing stderr outputs into contextual, language-accessible guidance without ever compromising the integrity of the evaluation process by leaking solutions. This approach directly addresses the cognitive over-reliance phenomenon identified in the literature, where premature exposure to complete solutions causes atrophy in students' independent algorithmic decomposition capabilities (Ma et al., 2025; Tsakeni et al., 2025).

The ablation finding that Role-Prompting alone contributes a 20% improvement in diagnostic accuracy has profound practical implications. It demonstrates that explicitly assigning the LLM a vocational educator persona (e.g., "You are a JavaScript tutor for SMK 11th-grade students...") activates domain-specific pedagogical reasoning pathways within the model's latent knowledge, enabling more contextually appropriate error diagnoses. For vocational educators seeking to implement AI-assisted programming instruction, this finding provides an actionable, low-complexity intervention: a well-crafted persona declaration in the system prompt is the single most impactful engineering decision.

The measured average latency of 4,895 ms, while functionally adequate for asynchronous feedback workflows, approaches the threshold of user-perceptible delay that may disrupt the interactive learning experience. Future implementations should consider Server-Sent Events (SSE) architectures to enable incremental streaming of JSON response packets, reducing perceived wait times below the 3-second interactive tolerance benchmark recommended by (Nielsen, 1993).

4. CONCLUSION

This experimental study demonstrated that systematic layered constraint prompt engineering can reliably transform a general-purpose Large Language Model into a pedagogically safe Socratic scaffolding tool for vocational programming education. Testing across 390 API calls to Gemini 2.5 Flash with 13 experimental configurations and 10 standardized JavaScript error scenarios yielded several definitive findings. The layered prompt architecture achieved a JSON Parsing Success Rate (PSR) of 99.2%, confirming that the combination of explicit JSON schema constraints, Few-Shot exemplars, and conservative temperature settings ($T = 0.4$) deterministically forces the language model to produce structurally valid output. The ablation study revealed that Role-Prompting the explicit declaration of a vocational educator persona, constitutes the single most influential prompt component, elevating error classification accuracy by 20 percentage points. Simultaneously, the prompt architecture achieved 100% defense against indirect prompt injection attacks, ensuring that the AI tutor cannot be manipulated by malicious student inputs to bypass its Socratic constraints. The strict zero-spoiler enforcement, restricting AI responses to a maximum of three graduated diagnostic hints without code solution exposure, operationalizes the theoretical construct of fading scaffolding support within an automated system. Collectively, these results establish that the triad of Role-Prompting, Schema Enforcement, and Quality Rules constitutes a robust and replicable prompt engineering framework for deploying LLM-based pedagogical agents in vocational education contexts, transforming cryptic compiler errors into guided, language-accessible learning opportunities that preserve the constructivist integrity of programming instruction.

Acknowledgments

Conflicts of Interest

The authors declare no conflict of interest regarding the publication of the paper.

References

- Alshammari, M. T. (2025). An Investigation into ChatGPT-Based Instructional Feedback on Programming Projects. *Proceedings of the 4th International Conference on Computing and Information Technology (ICCIT)*, IEEE, 624–628.
<https://doi.org/10.1109/ICCIT63348.2025.10989326>
- Chidambaram, S., Li, E. L., Bai, M., & Williams, A. C. (2024). Socratic Human Feedback (SoHF): Expert Steering Strategies for LLM Code Generation. *Findings of the Association for Computational Linguistics (EMNLP 2024)*, 15491–15502.
<https://doi.org/10.18653/v1/2024.findings-emnlp.908>
- Frankford, E., Cikalleshi, E., & Breu, R. (2026). Chatbot-Based Assessment of Code Understanding in Automated Programming Assessment Systems. *ArXiv Preprint Software Engineering*. <https://arxiv.org/abs/2604.07304>
- Greshake, K., & others. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 79–90.
<https://arxiv.org/abs/2302.12173>
- Ibañez, R. (2023). Impact of Online Gaming on the Academic Performance of DEBESMSCAT-Cawayan Campus Students. *Scientific Journal of Informatics*, 10(4), 423–434.
<https://doi.org/10.15294/sji.v10i4.45007>

- Liao, J., Zhong, L., Zhe, L., Xu, H., Liu, M., & Xie, T. (2024). Scaffolding Computational Thinking With ChatGPT. *IEEE Transactions on Learning Technologies*, 17, 1628–1642.
<https://doi.org/10.1109/TLT.2024.3392896>
- Ma, S., Wang, J., Zhang, Y., Ma, X., & Wang, A. Y. (2025). DBox: Scaffolding Algorithmic Programming Learning through Learner-LLM Co-Decomposition. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 1–20.
<https://doi.org/10.1145/3706598.3713748>
- Nielsen, J. (1993). *Usability Engineering*. Academic Press.
- Schulhoff, S., & others. (2024). The Prompt Report: A Systematic Survey of Prompting Techniques. *ArXiv Preprint ArXiv:2406.06608*. <https://arxiv.org/abs/2406.06608>
- Tsakeni, M., Nwafor, S. C., Mosia, M., & Egara, F. O. (2025). Mapping the Scaffolding of Metacognition and Learning by AI Tools in STEM Classrooms: A Bibliometric–Systematic Review Approach (2005–2025). *Journal of Intelligence*, 13(11), 148.
<https://doi.org/10.3390/jintelligence13110148>
- Yilmaz, R., & Yilmaz, F. G. K. (2023). The Effect of Generative Artificial Intelligence (AI)-Based Tool Use on Students' Computational Thinking Skills, Programming Self-Efficacy and Motivation. *Computers and Education: Artificial Intelligence*, 4, 12–25.
<https://doi.org/10.1016/j.caeai.2023.100147>
- Zhang, Z., Dong, Z., Shi, Y., Price, T., Matsuda, N., & Xu, D. (2024). Students' Perceptions and Preferences of Generative Artificial Intelligence Feedback for Programming. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(21), 23250–23258.
<https://doi.org/10.1609/aaai.v38i21.30372>